



## دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

## درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

## INDEX

|                                      |    |
|--------------------------------------|----|
| Imports .....                        | 3  |
| Prepare Dataset.....                 | 3  |
| Build Decision Tree .....            | 4  |
| Entropy Calculation .....            | 4  |
| Information Gain calculation.....    | 5  |
| Build Decision Tree recursively..... | 7  |
| Print Tree Structure .....           | 9  |
| Plot Decision Tree .....             | 9  |
| Analysis and Result .....            | 10 |
| Conclusion .....                     | 11 |

## Imports

```
from math import log2
```

این دستور تابع لگاریتم در مبنای ۲ را وارد می‌کند که ابزار اصلی برای محاسبه آنتروپی شانون است؛ از آن برای سنجش میزان عدم قطعیت یا آشفتگی در داده‌ها استفاده می‌شود.

```
import pandas as pd
```

از این کتابخانه، برای تبدیل داده‌ی خام به دیتا فریم استفاده شده است.

```
import matplotlib.pyplot as plt
```

از این کتابخانه برای ترسیم درخت استفاده شده است.

## Prepare Dataset

```
columns = ['f1', 'f2', 'f3', 'f4', 'Y']
data = [
    [0, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    ...,
    [0, 0, 0, 1, 0],
    [0, 1, 0, 1, 1]
]
df = pd.DataFrame(data, columns=columns)
```

در این بخش کد، با استفاده از کتابخانه Pandas یک object از نوع DataFrame ایجاد می‌شود.

- data به عنوان داده‌های اصلی جدول استفاده می‌شود.
- columns=columns نام ستون‌های f1 تا Y را که قبلاً تعریف شده‌اند به این داده‌ها اختصاص می‌دهد.

در نتیجه متغیر df شامل یک DataFrame می‌شود که ۱۶ سطر و ۵ ستون دارد.

## Entropy Calculation 🚦

آنتروپی میزان عدم قطعیت یا بی‌نظمی در داده‌ها را اندازه‌گیری می‌کند. هدف این تابع محاسبه آنتروپی توزیع برچسب‌ها است.

```
def entropy(labels):
```

ورودی این تابع، ستون پنجم متغیر df یا همان labels است که شامل برچسب‌ها می‌باشد.

```
p1 = sum(labels) / len(labels)
```

در این خط، مقدار احتمال رخداد کلاس ۱ محاسبه می‌شود.

- sum(labels) مجموع مقادیر موجود در لیست labels را محاسبه می‌کند.
- چون مقادیر برچسب‌ها ۰ یا ۱ هستند، جمع آن‌ها برابر با تعداد نمونه‌های کلاس ۱ خواهد بود.
- len(labels) تعداد کل نمونه‌ها را به دست می‌آورد.

$$p_1 = \frac{\text{تعداد نمونه‌های کلاس 1}}{\text{کل نمونه‌ها}}$$

بنابراین مقدار p1 برابر است با:

که همان احتمال وقوع کلاس ۱ در داده‌ها است.

```
if p1 == 0 or p1 == 1:
    return 0
```

در این بخش بررسی می‌شود که آیا توزیع برچسب‌ها کاملاً یکنواخت است یا خیر. دو حالت بررسی می‌شود:

اگر  $p1 = 0$  باشد یعنی همه نمونه‌ها کلاس ۰ هستند و اگر  $p1 = 1$  باشد یعنی همه نمونه‌ها کلاس ۱ هستند.

در این شرایط هیچ عدم قطعیتی در داده وجود ندارد، بنابراین مقدار آنتروپی برابر صفر خواهد بود. زیرا توزیع کاملاً قطعی است.

```
p0 = 1 - p1
```

در این خط احتمال کلاس 0 محاسبه می‌شود. چون تنها دو کلاس وجود دارد (۰ و ۱)، مجموع احتمالات باید برابر ۱ باشد

```
return -(p1 * log2(p1) + p0 * log2(p0))
```

در این خط مقدار آنتروپی بر اساس فرمول استاندارد آنتروپی شانون محاسبه و بازگردانده می‌شود. فرمول آنتروپی برای یک متغیر دودویی به صورت زیر است:

$$H(P) = - \sum_{x \in C} P(x) \log P(x)$$

## Information Gain calculation 🚦

هدف این تابع محاسبه Information Gain برای یک ویژگی مشخص است. این معیار در الگوریتم‌های درخت تصمیم برای انتخاب بهترین ویژگی جهت تقسیم داده‌ها استفاده می‌شود.

```
def information_gain(df, feature):
```

این تابع دو ورودی دریافت می‌کند:

- df : DataFrame که قبلاً ساختیم
- feature : نام یکی از ۴ ویژگی‌های موجود در داده‌ها که قرار است میزان اهمیت آن در پیش‌بینی متغیر هدف بررسی شود.

```
total_entropy = entropy(df['Y'])
```

در این خط آنتروپی کل داده‌ها محاسبه می‌شود.

- df['Y'] ستون برچسب‌ها (متغیر هدف) را از DataFrame استخراج می‌کند.
- تابع entropy که قبلاً تعریف شده است، روی این برچسب‌ها اعمال می‌شود.

بنابراین total\_entropy بیانگر میزان عدم قطعیت در کل مجموعه داده قبل از انجام هرگونه تقسیم‌بندی است.

```
weighted_entropy = 0
```

یک متغیر به نام weighted\_entropy تعریف شده و مقداردهی اولیه می‌شود. این متغیر برای نگهداری مجموع آنتروپی‌های وزندهی شده زیرمجموعه‌ها استفاده خواهد شد. (پس از تقسیم داده‌ها بر اساس ویژگی موردنظر، آنتروپی هر بخش محاسبه شده و با توجه به اندازه آن بخش وزندهی می‌شود).

```
for value in df[feature].unique():
```

در این خط یک حلقه for تعریف شده است که روی تمام مقادیر ویژگی مورد نظر تکرار می‌کند.

- df[feature] ستون مربوط به ویژگی مورد نظر را استخراج می‌کند.
- unique () تمام مقادیر متمایز آن ویژگی را برمی‌گرداند. (که در اینجا ۰ و ۱ هست)

هدف این حلقه بررسی تمام حالت‌های ممکن تقسیم داده‌ها بر اساس آن ویژگی است.

```
subset = df[df[feature] == value]
```

subset یک زیرمجموعه از داده‌هاست. subset شامل تمام نمونه‌هایی است که در آن‌ها ویژگی ورودی، مقدار value را دارد.

```
subset_entropy = entropy(subset['Y'])
```

در این خط آنتروپی لیبل های موجود در زیرمجموعه محاسبه می شود.

- subset['Y'] ستون لیبل ها در زیرمجموعه را استخراج می کند.
- تابع entropy میزان عدم قطعیت برجسب ها در این بخش از داده ها را محاسبه می کند.

این مقدار نشان می دهد که پس از تقسیم داده ها بر اساس این ویژگی، زیرمجموعه موردنظر چقدر خالص یا نامنظم است.

```
weighted_entropy += (len(subset) / len(df)) * subset_entropy
```

در این خط آنتروپی وزن دهی شده زیرمجموعه به مجموع کل اضافه می شود.

$$Weight = \frac{|subset|}{|dataset|}$$

len(subset) تعداد نمونه های موجود در زیرمجموعه  
len(df) تعداد کل نمونه های مجموعه داده

نسبت این دو مقدار نشان دهنده وزن یا سهم زیرمجموعه در کل داده ها است . به این ترتیب، مجموع آنتروپی های تمام زیرمجموعه ها با در نظر گرفتن اندازه آن ها محاسبه می شود.

```
return total_entropy - weighted_entropy
```

در این خط مقدار Information Gain محاسبه و بازگردانده می شود. در واقع این مقدار نشان می دهد که دانستن مقدار یک ویژگی تا چه حد عدم قطعیت درباره متغیر هدف را کاهش می دهد.

## Build Decision Tree recursively 🚀

این تابع، هسته اصلی الگوریتم درخت تصمیم را پیاده‌سازی می‌کند. این تابع به صورت بازگشتی عمل کرده و ساختار درختی را برای داده‌ها ایجاد می‌کند.

```
def build_tree(df, features, depth=0):
```

این خط، تابع `build_tree` را با ورودی‌های داده (`df`)، لیست ویژگی‌ها (`features`) و عمق فعلی (`depth`) تعریف می‌کند.

```
node_entropy = entropy(df['Y'])
```

آنتروپی برچسب‌های موجود در گره فعلی محاسبه می‌شود.

```
if df['Y'].nunique() == 1:
    label = int(df['Y'].iloc[0])
    return label
```

اگر تنها یک کلاس در گره وجود داشته باشد، گره کاملاً خالص است. در این حالت:

- گره به یک برگ نهایی تبدیل می‌شود.
- همان برچسب، خروجی گره است.

این یکی از شرایط توقف استاندارد در الگوریتم ID3 است.

```
if not features:
    label = int(df['Y'].mode()[0])
    return label
```

اگر هیچ ویژگی‌ای برای تقسیم باقی نمانده باشد:

- از رأی اکثریت در میان کلاس‌ها استفاده می‌شود.
- این روش مانع از شکست الگوریتم هنگامی می‌شود که آنتروپی صفر نشده اما امکان تقسیم دیگر وجود ندارد.

```
best_feature = None
best_gain = -1
```

دو متغیر برای ذخیره بهترین ویژگی با بیشترین IG مقداردهی اولیه می‌شوند.

```
for f in features:
    gain = information_gain(df, f)
    print(f"{indent} IG({f}) = {gain:.4f}")
```

برای هر ویژگی، مقدار IG محاسبه و چاپ می‌شود.

```

if gain > best_gain:
    best_gain = gain
    best_feature = f

```

ویژگی ای که بالاترین IG را دارد، به عنوان بهترین ویژگی انتخاب می شود.

```

tree = {best_feature: {}}
remaining = [f for f in features if f != best_feature]

```

یک دیکشنری برای نگهداری شاخه های درخت ساخته می شود. مثلاً `{'f1': {0: ..., 1: ...}}` ویژگی انتخاب شده از فهرست ویژگی های باقی مانده حذف می شود تا در سطح پایین تر دوباره استفاده نشود.

```

for value in sorted(df[best_feature].unique()):

```

برای هر مقدار ممکن از ویژگی، این حلقه یکبار تکرار می شود:

```

subset = df[df[best_feature] == value]
tree[best_feature][int(value)] = build_tree(subset, remaining, depth + 1)

```

مراحل:

۱. داده های گره بر اساس مقدار ویژگی به زیرمجموعه تقسیم می شود.
۲. تابع `build_tree` به صورت بازگشتی برای هر زیرمجموعه فراخوانی می شود.
۳. عمق گره یک واحد افزایش می یابد.
۴. ساختار درخت به صورت دیکشنری تو در تو ساخته می شود.

این دقیقاً همان گسترش بازگشتی درخت تصمیم است.

```

return tree

```

در پایان، ساختار کامل گره به همراه زیرشاخه هایش بازگردانده می شود.



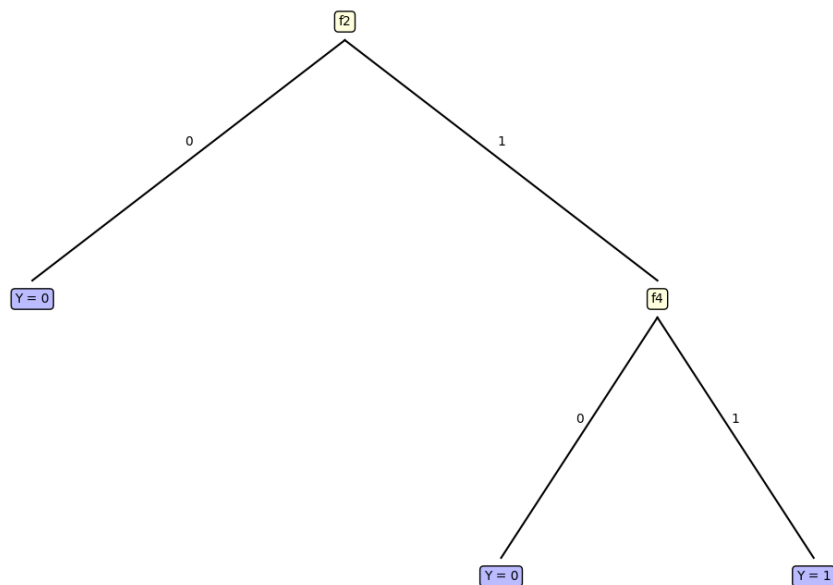
## Print Tree Structure

این کد برای نمایش و ساخت درخت تصمیم استفاده می‌شود. تابع `print_tree` یک درخت تصمیم که به صورت دیکشنری ذخیره شده است را به شکل قابل خواندن چاپ می‌کند. اگر مقدار ورودی دیکشنری نباشد، یعنی به گره نهایی (برگ) رسیده‌ایم و مقدار خروجی `Y` چاپ می‌شود. در غیر این صورت، نام ویژگی از کلید دیکشنری گرفته می‌شود و برای هر مقدار ممکن آن ویژگی، شرطی به صورت `if feature == value` چاپ می‌شود و سپس تابع به صورت بازگشتی برای زیردرخت مربوط به آن مقدار اجرا می‌شود تا کل ساختار درخت نمایش داده شود.

در بخش پایانی، ابتدا آنتروپی کل دیتاست با تابع `entropy` محاسبه و چاپ می‌شود، سپس با استفاده از ویژگی‌های `f1` تا `f4` درخت تصمیم توسط تابع `build_tree` ساخته می‌شود و در نهایت ساختار کامل درخت تصمیم با فراخوانی `print_tree(tree)` به صورت مرحله‌ای چاپ می‌گردد.

## Plot Decision Tree

این تابع برای رسم گرافیکی درخت تصمیم با استفاده از کتابخانه `matplotlib` طراحی شده است. ابتدا یک شکل (`figure`) و محور (`axis`) ایجاد می‌شود و خطوط و مقیاس‌های محور غیرفعال می‌شوند تا فقط ساختار درخت دیده شود. سپس تابع داخلی `draw_node` به صورت بازگشتی گره‌های درخت را رسم می‌کند. اگر گره یک دیکشنری نباشد، به عنوان گره برگ در نظر گرفته شده و مقدار خروجی `Y` نمایش داده می‌شود. در غیر این صورت نام ویژگی به ویژگی به عنوان گره تصمیم رسم می‌شود و برای هر شاخه‌ی آن ویژگی، موقعیت افقی و عمودی فرزندان محاسبه می‌شود، یک خط بین گره والد و فرزند رسم می‌شود و مقدار شاخه روی خط نوشته می‌شود. سپس همین فرآیند برای زیردرخت‌ها به صورت بازگشتی ادامه پیدا می‌کند تا کل درخت رسم شود. در نهایت رسم درخت از ریشه شروع شده، تصویر در فایل `tree_plot.png` ذخیره می‌شود و سپس نمایش داده می‌شود.



F2 and F4

===== DATASET ENTROPY =====

Entropy(D) = 0.8113

===== BUILDING DECISION TREE =====

Node depth 0

Samples: 16

Entropy: 0.8113

Information Gain:

IG(f1) = 0.0000

IG(f2) = 0.3113

IG(f3) = 0.0000

IG(f4) = 0.3113

Selected feature → f2

Split: f2 = 0

Node depth 1

Samples: 8

Entropy: 0.0000

Leaf node → Y = 0

Split: f2 = 1

Node depth 1

Samples: 8

Entropy: 1.0000

Information Gain:

IG(f1) = 0.0000

IG(f3) = 0.0000

IG(f4) = 1.0000

Selected feature → f4

Split: f4 = 0

Node depth 2

Samples: 4

Entropy: 0.0000

Leaf node → Y = 0

Split: f4 = 1

Node depth 2

Samples: 4

Entropy: 0.0000

Leaf node → Y = 1

===== FINAL DECISION TREE =====

if f2 == 0:

→ Y = 0

if f2 == 1:

if f4 == 0:

→ Y = 0

if f4 == 1:

→ Y = 1

آنتروپی کل دیتاست برابر با ۰.۸۱۱۳ محاسبه شد که نشان می‌دهد برچسب‌های کلاس در داده‌ها تا حدی با هم مخلوط هستند و عدم قطعیت نسبتاً متوسطی در مجموعه داده وجود دارد. برای ساخت درخت تصمیم، Information Gain هر ویژگی محاسبه شد. در گره ریشه، ویژگی‌های  $f_2$  و  $f_4$  بیشترین مقدار Information Gain یعنی ۰.۳۱۱۳ را داشتند، در حالی که  $f_1$  و  $f_3$  هیچ کاهش قابل توجهی در آنتروپی ایجاد نکردند. بنابراین ویژگی  $f_2$  به عنوان اولین معیار تقسیم در ریشه درخت انتخاب شد.

پس از تقسیم داده‌ها بر اساس  $f_2$ ، شاخه‌ای که در آن  $f_2 = 0$  بود به یک گره کاملاً خالص با آنتروپی ۰ تبدیل شد، به این معنی که همه نمونه‌ها دارای برچسب  $Y = 0$  بودند و بنابراین یک گره برگ ایجاد شد. در شاخه‌ای که  $f_2 = 1$  بود، آنتروپی برابر ۱.۰ باقی ماند که نشان‌دهنده توزیع کاملاً مخلوط کلاس‌ها است. در این مرحله، از بین ویژگی‌های باقی‌مانده،  $f_4$  با مقدار Information Gain برابر ۱.۰ بهترین انتخاب برای تقسیم بعدی بود. این تقسیم داده‌ها را به طور کامل جدا کرد: زمانی که  $f_4 = 0$  باشد خروجی  $Y = 0$  و زمانی که  $f_4 = 1$  باشد خروجی  $Y = 1$  خواهد بود. در نتیجه درخت نهایی تنها شامل دو سطح تصمیم‌گیری است و توانسته داده‌ها را به طور کامل تفکیک کند.

## Conclusion

نتایج نشان می‌دهد که الگوریتم درخت تصمیم توانست مجموعه داده را تنها با استفاده از دو ویژگی  $f_2$  و  $f_4$  به طور مؤثر طبقه‌بندی کند. اگرچه در ابتدا چهار ویژگی در اختیار الگوریتم قرار داشت، اما الگوریتم با استفاده از معیار Information Gain به طور خودکار ویژگی‌هایی را انتخاب کرد که بیشترین تأثیر را در کاهش عدم قطعیت داشتند. ویژگی‌های  $f_1$  و  $f_3$  هیچ تأثیر قابل توجهی در کاهش آنتروپی نداشتند، بنابراین در مدل نهایی مورد استفاده قرار نگرفتند. این موضوع یکی از مزایای مهم درخت‌های تصمیم را نشان می‌دهد، زیرا این الگوریتم‌ها به طور ضمنی عمل Feature Selection را نیز انجام می‌دهند.

همچنین مشاهده می‌شود که درخت ساخته شده بسیار ساده و کم‌عمق است و حداکثر عمق آن تنها دو سطح است. درخت‌های ساده‌تر معمولاً تفسیرپذیری بیشتری دارند و درک قوانین تصمیم‌گیری آن‌ها آسان‌تر است، که این موضوع یکی از مزایای مهم مدل‌های درخت تصمیم نسبت به بسیاری از الگوریتم‌های پیچیده‌تر یادگیری ماشین محسوب می‌شود.

در این مثال، قوانین تصمیم‌گیری به شکل بسیار ساده قابل بیان هستند: اگر  $f_2 = 0$  باشد خروجی همیشه ۰ است، و اگر  $f_2 = 1$  باشد، پیش‌بینی تنها به مقدار  $f_4$  بستگی دارد.

به طور کلی، این آزمایش نشان می‌دهد که معیارهای Entropy و Information Gain چگونه فرآیند ساخت درخت تصمیم را هدایت می‌کنند و تقسیم‌هایی را انتخاب می‌کنند که بیشترین کاهش در عدم قطعیت را ایجاد کنند. مدل نهایی توانست تمام نمونه‌های موجود در دیتاست را به درستی طبقه‌بندی کند، که نشان می‌دهد ویژگی‌های انتخاب‌شده اطلاعات کافی برای تمایز بین کلاس‌ها را فراهم می‌کنند. این مثال به خوبی کارایی و قابلیت تفسیر بالای درخت‌های تصمیم را در مجموعه داده‌های کوچک و ساختاریافته نشان می‌دهد.